

EXPERIMENT-33

Algorithm:

1. Start.

2. Display the menu:

Verify associative property of matrix addition.

Verify associative property of matrix multiplication.

Exit.

3. Read the user's choice.

4. For matrix addition:

Input m and n.

Declare three VLA matrices A, B, and C of size $m \times n$.

Read all three matrices.

Calculate $A + B$.

Calculate $(A + B) + C$.

Calculate $B + C$.

Calculate $A + (B + C)$.

Display both final matrices.

Compare them.

If equal, associative property is verified.

5. For matrix multiplication:

Input m, n, p, and q.

Declare:

A as $m \times n$

B as $n \times p$

C as $p \times q$

Calculate $A \times B$, which has size $m \times p$.

Calculate $(A \times B) \times C$, which has size $m \times q$.

Calculate $B \times C$, which has size $n \times q$.

Calculate $A \times (B \times C)$, which has size $m \times q$.

Display both final matrices.

Compare them.

If equal, associative property is verified.

6. Repeat the menu until the user chooses Exit.

7. Stop.

Important dimensions for multiplication

Operation	Dimensions	Result
$A \times B$	$(m \times n)(n \times p)$	$m \times p$
$(A \times B) \times C$	$(m \times p)(p \times q)$	$m \times q$
$B \times C$	$(n \times p)(p \times q)$	$n \times q$
$A \times (B \times C)$	$(m \times n)(n \times q)$	$m \times q$

Therefore, both sides produce an $m \times q$ matrix, allowing us to compare them and verify:

$$(A \times B) \times C = A \times (B \times C)$$

Program, Compilation and Execution:

```
#include <stdio.h>

// Function to read a matrix
void readMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

// Function to display a matrix
void displayMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}

// Function to compare two matrices
int compareMatrix(int r, int c, int a[r][c], int b[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            if (a[i][j] != b[i][j])
            {
                return 0;
            }
        }
    }
}
```

```
}  
}  
  
    return 1;  
}  
  
// Function to verify associative property of matrix addition  
void associativeAddition()  
{  
    int m, n, i, j;  
  
    printf("\nEnter number of rows and columns (m n): ");  
    scanf("%d %d", &m, &n);  
  
    int A[m][n], B[m][n], C[m][n];  
    int AB[m][n], BC[m][n];  
    int left[m][n], right[m][n];  
  
    printf("Enter elements of matrix A:\n");  
    readMatrix(m, n, A);  
  
    printf("Enter elements of matrix B:\n");  
    readMatrix(m, n, B);  
  
    printf("Enter elements of matrix C:\n");  
    readMatrix(m, n, C);  
  
    // Calculate A + B  
    for (i = 0; i < m; i++)  
    {  
        for (j = 0; j < n; j++)  
        {  
            AB[i][j] = A[i][j] + B[i][j];  
        }  
    }  
  
    // Calculate (A + B) + C  
    for (i = 0; i < m; i++)  
    {  
        for (j = 0; j < n; j++)  
        {  
            left[i][j] = AB[i][j] + C[i][j];  
        }  
    }  
}
```

```

// Calculate B + C
for (i = 0; i < m; i++)
{
    for (j = 0; j < n; j++)
    {
        BC[i][j] = B[i][j] + C[i][j];
    }
}

// Calculate A + (B + C)
for (i = 0; i < m; i++)
{
    for (j = 0; j < n; j++)
    {
        right[i][j] = A[i][j] + BC[i][j];
    }
}

printf("\n(A + B) + C =\n");
displayMatrix(m, n, left);

printf("\nA + (B + C) =\n");
displayMatrix(m, n, right);

if (compareMatrix(m, n, left, right))
{
    printf("\n(A + B) + C = A + (B + C)\n");
    printf("Associative property of matrix addition is verified.\n");
}
else
{
    printf("\nAssociative property is not verified.\n");
}
}

// Function to verify associative property of matrix multiplication
void associativeMultiplication()
{
    int m, n, p, q;
    int i, j, k;

    printf("\nEnter m, n, p and q: ");
    scanf("%d %d %d %d", &m, &n, &p, &q);

    int A[m][n];
    int B[n][p];
    int C[p][q];

```

```

int AB[m][p];
int BC[n][q];

int left[m][q];
int right[m][q];

printf("Enter elements of matrix A (%d x %d):\n", m, n);
readMatrix(m, n, A);

printf("Enter elements of matrix B (%d x %d):\n", n, p);
readMatrix(n, p, B);

printf("Enter elements of matrix C (%d x %d):\n", p, q);
readMatrix(p, q, C);

// Calculate A x B
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
    {
        AB[i][j] = 0;
        for (k = 0; k < n; k++)
        {
            AB[i][j] += A[i][k] * B[k][j];
        }
    }
}

// Calculate (A x B) x C
for (i = 0; i < m; i++)
{
    for (j = 0; j < q; j++)
    {
        left[i][j] = 0;
        for (k = 0; k < p; k++)
        {
            left[i][j] += AB[i][k] * C[k][j];
        }
    }
}

```

```
// Calculate B x C
for (i = 0; i < n; i++)
{
    for (j = 0; j < q; j++)
    {
        BC[i][j] = 0;
        for (k = 0; k < p; k++)
        {
            BC[i][j] += B[i][k] * C[k][j];
        }
    }
}

// Calculate A x (B x C)
for (i = 0; i < m; i++)
{
    for (j = 0; j < q; j++)
    {
        right[i][j] = 0;
        for (k = 0; k < n; k++)
        {
            right[i][j] += A[i][k] * BC[k][j];
        }
    }
}

printf("\n(A x B) x C =\n");
displayMatrix(m, q, left);

printf("\nA x (B x C) =\n");
displayMatrix(m, q, right);

if (compareMatrix(m, q, left, right))
{
    printf("\n(A x B) x C = A x (B x C)\n");
    printf("Associative property of matrix multiplication is verified.\n");
}
else
{
    printf("\nAssociative property is not verified.\n");
}
}
```

```
GNU nano 9.0 33.c
{
    printf("\n(A x B) x C = A x (B x C)\n");
    printf("Associative property of matrix multiplication is verified.\n");
}
else
{
    printf("\nAssociative property is not verified.\n");
}
}

int main()
{
    int choice;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Verify associative property of matrix addition\n");
        printf("2. Verify associative property of matrix multiplication\n");
        printf("3. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                associativeAddition();
                break;

            case 2:
                associativeMultiplication();
                break;

            case 3:
                printf("Exiting program... \n");
                break;

            default:
                printf("Invalid choice! Please enter 1, 2 or 3.\n");
        }

    } while (choice != 3);

    return 0;
}
```

```
===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1

Enter number of rows and columns (m n): 2
3
Enter elements of matrix A:
1 2
3 4
5 6
Enter elements of matrix B:
7 8
9 10
11 12
Enter elements of matrix C:
13 14
15 16
17 18

(A + B) + C =
21    24    27
30    33    36

A + (B + C) =
21    24    27
30    33    36

(A + B) + C = A + (B + C)
Associative property of matrix addition is verified.
```

```

      MENU
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 2

Enter m, n, p and q: 2 3 3 2
Enter elements of matrix A (2 x 3):
1 2 3
4 5 6
Enter elements of matrix B (3 x 3):
1 2 3
4 5 6
7 8 9
Enter elements of matrix C (3 x 2):
1 2
3 4
5 6

(A x B) x C =
348    456
789    1032

A x (B x C) =
348    456
789    1032

(A x B) x C = A x (B x C)
Associative property of matrix multiplication is verified.
```

NAME: TAANISH AGARWAL
BATCH: B-15
SAP ID: 590034662